

Module 3: Machine Learning

Feature Engineering

Agenda

- Feature Selection & Engineering
- Math free overview of classification models
- Evaluating Model Performance
- Improving model performance

Isn't Machine Learning Obsolete?

Can't We Just Use AI for Analysis?

Can't We Just Use AI to Build ML Models?

NO!

Determinism vs. Non-Determinism

- Deterministic functions will always produce the same output for the same input.
- Non-Deterministic functions will not.
- LLMs/AI by their nature are non-deterministic and cannot be guaranteed to produce the same output given the consistent input.

If you require guaranteed consistency, you can't use AI.

If you need to audit your results, you can't use AI.

Better Approach: Use AI to generate intermediate analytic code.

AI can code a ML model, but it can't tune one.

AI is very expensive and slow.


butter knife

The Machine Learning Process

Machine Learning

- **Supervised Learning:** Supervised Learning is a class of Machine Learning in which a model is "trained" using a set of pre-existing labeled data.

- **Unsupervised Learning:** A class of Machine Learning algorithms in which a model is built without the use of labeled data.

Machine Learning Problem Types

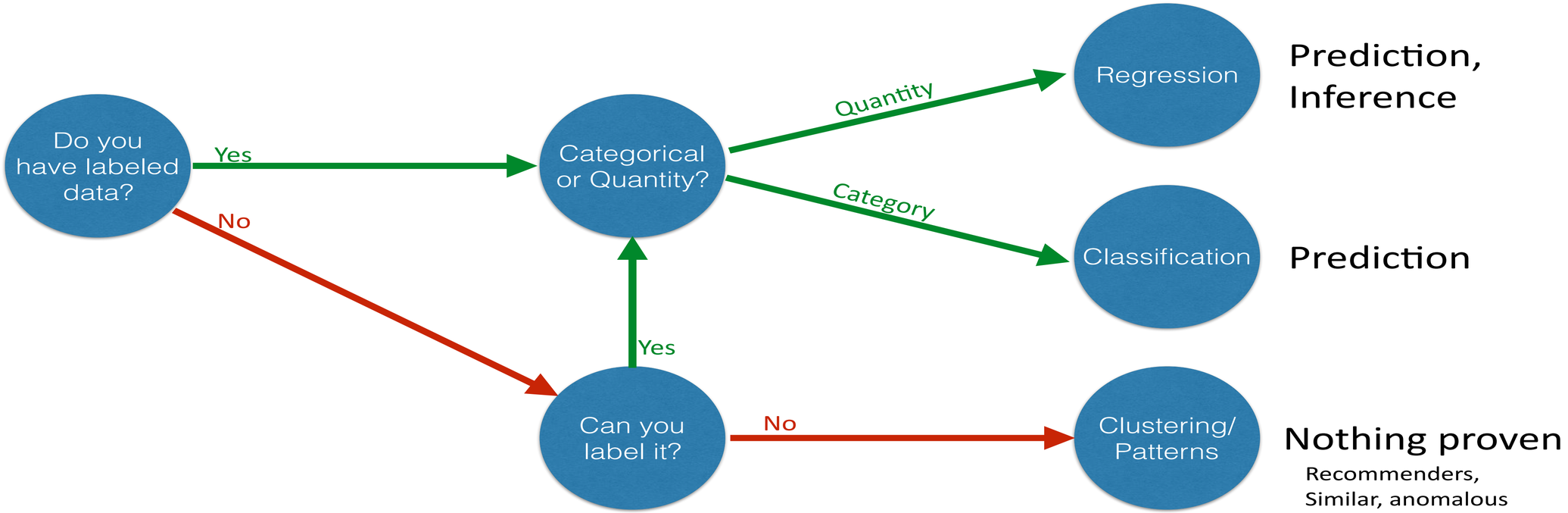
- **Classification**: Assigning or predicting a observation's membership in discrete class
- **Regression:** Predicting a continuous value based on the observations' features

- **Clustering:** Identifying groupings within a dataset
- **Dimensionality Reduction:** Reducing the number of variables in a feature set

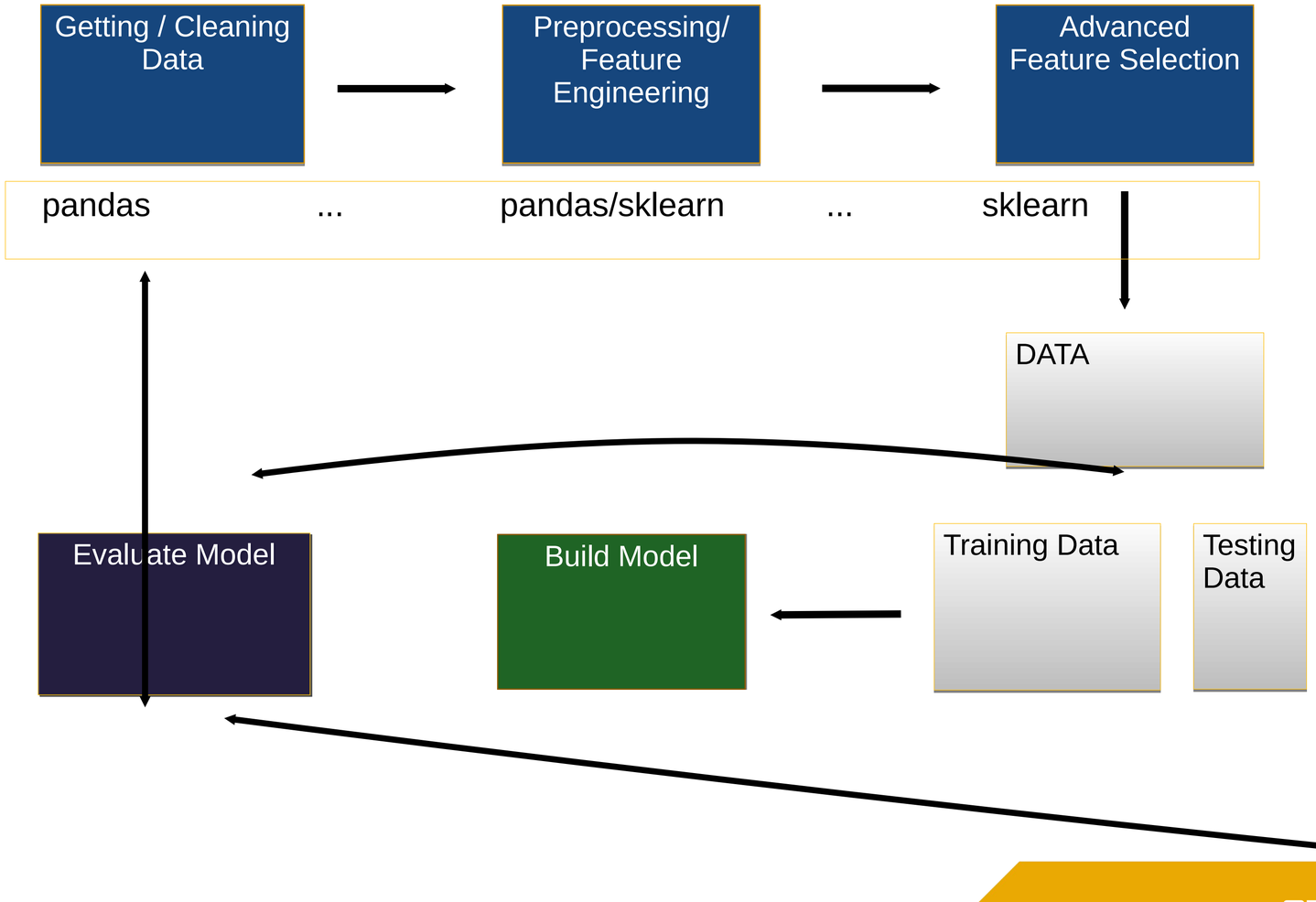
Machine Learning Terms

- **Features:** The mathematical representation of the original data. The features are the columns in your data set. Since the features will be a matrix, the are often written as X.
- **Observations:** The rows of your feature set.
- **Target:** The variable that you are trying to predict. Often represented as y.

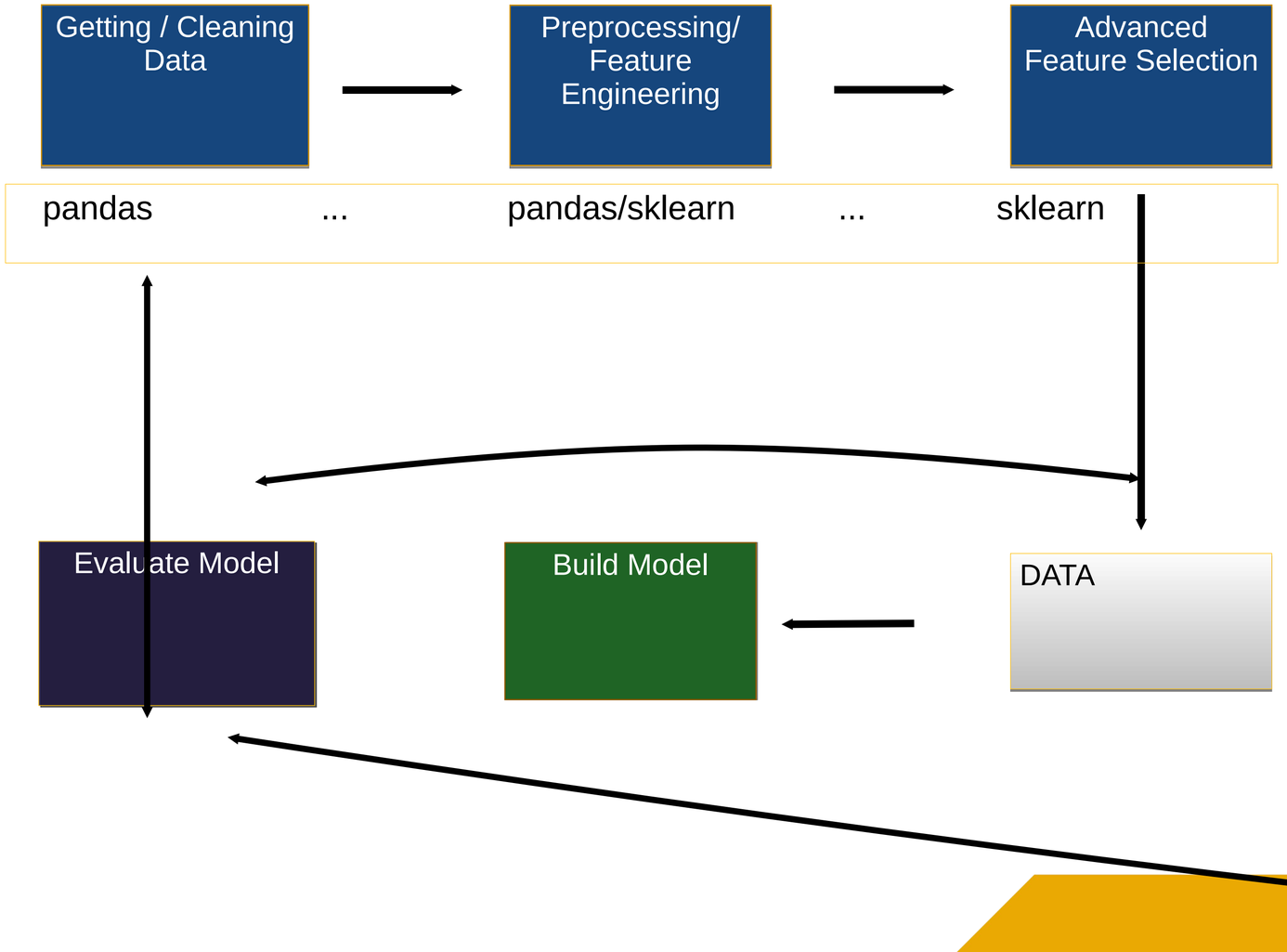
What Data Science kind of is…



Supervised Machine Learning Process



Unsupervised Machine Learning Process



How do you define success?
What are you measuring?

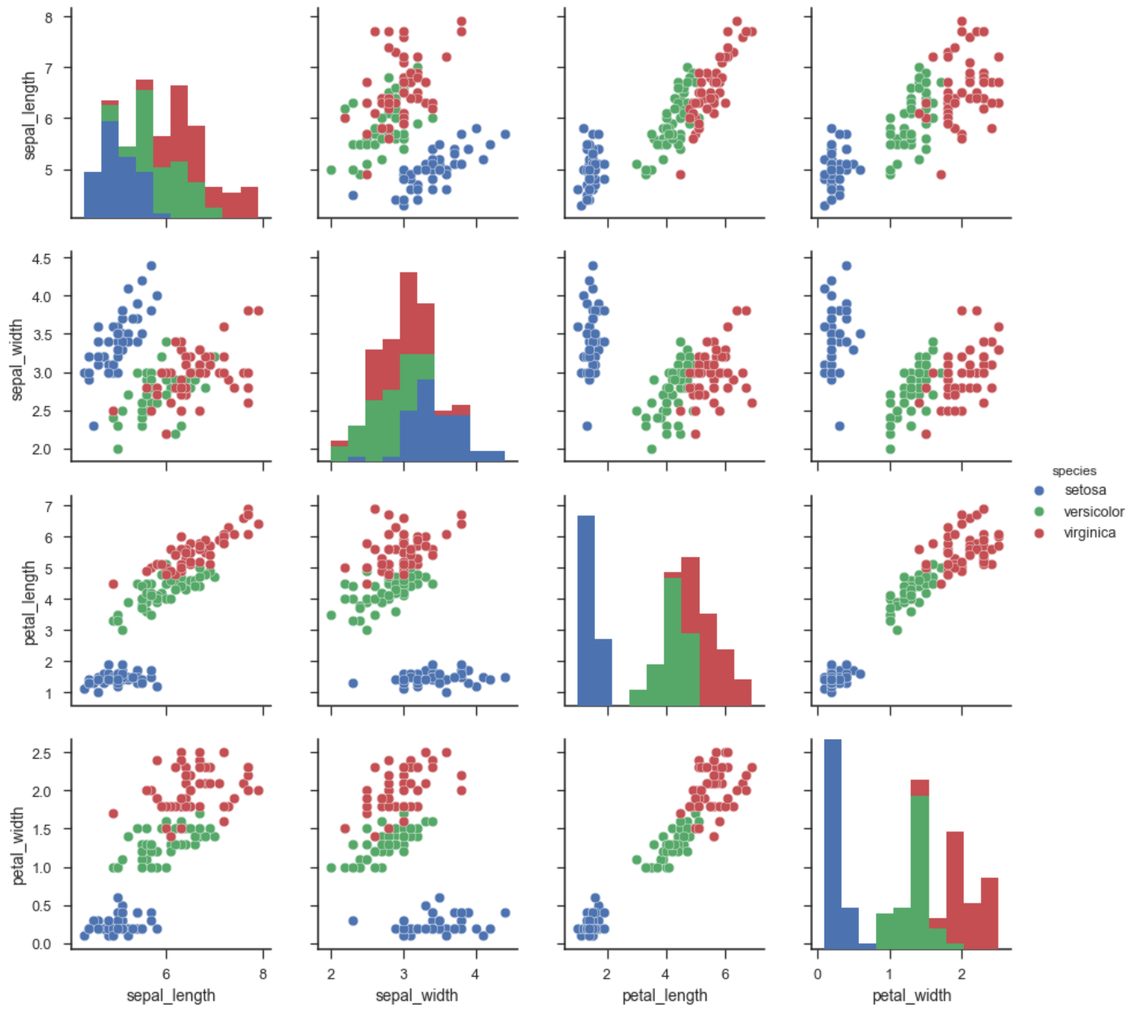
Choose data sources

- What is available?
- Is it enough?
- Is the data reliable/clean/consistent?
- What other data could you use?

Other Considerations

- Policies
- Legal contraints
- Biases in Data
- Latency
- Data size

Gather and Explore Your Data



Is the data good enough?

What are the rules governing its use?

Do I have enough?

Do problems or biases exist in the data that could cause problems?

Feature Engineering

Feature Engineering

- Define what you are trying to measure. These will become the **observations** or rows of your final dataset
- Define how you will mathematically represent your data. This will be come the **features** or columns of your final dataset.

Features

http://www.google.com





Features

http://www.google.com



Representation of URL Knowledge

- Come up with a **representation/set of knowledge** that has **enough complexity** to accurately describe the problem for the computer
- Knowledge here does not mean hard-coded knowledge or formal set of rules
- **The computer rather uses the knowledge we provide to extract patterns and acquire own knowledge**
- We should provide knowledge about reality that has **high variance about the problem** it describes (e.g. a feature that is high when it rains and low when it’s sunshine)

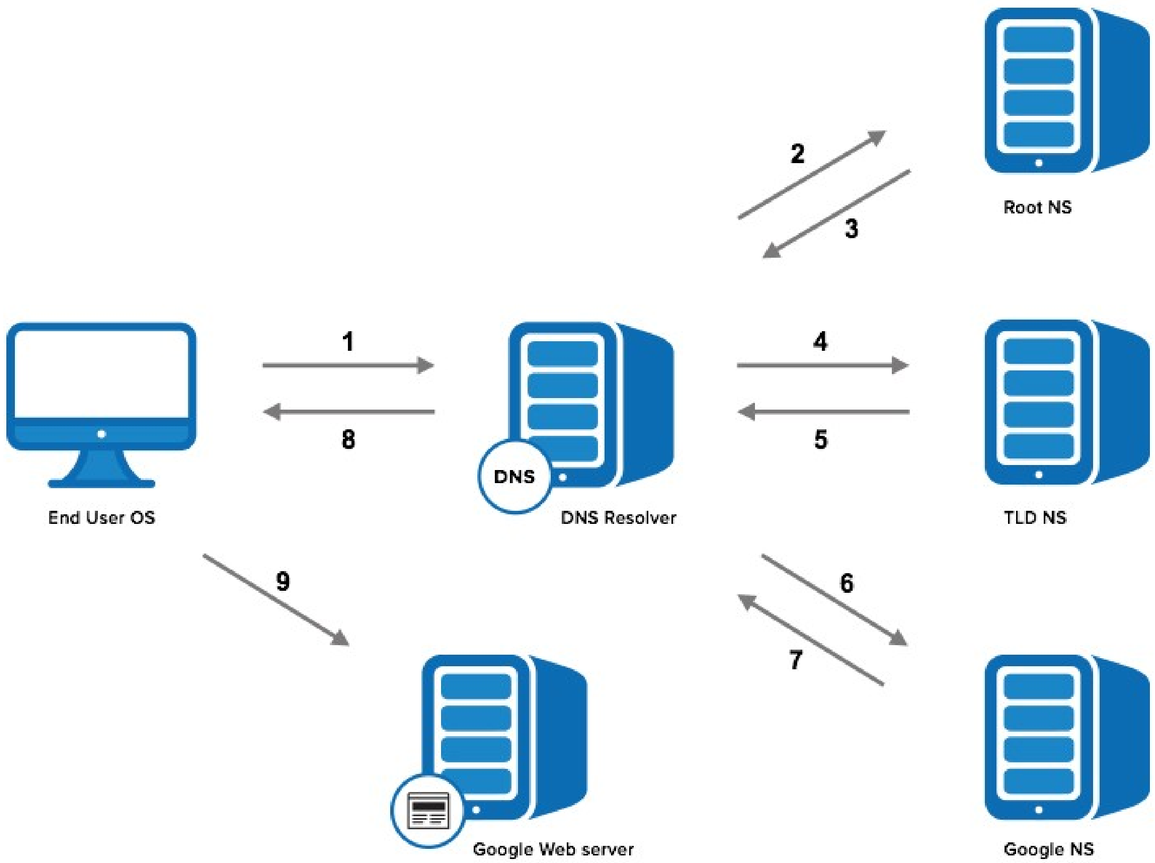
https://www.google.com/search?q=URL&source=lnms&tbm=isch&sa=X&ved=0ahUKEwjcl6ut-IDUAhVEPCYKHdJGDsYQ_AUIDCgD&biw=1215&bih=652

DNS 101

- **Domain Name Service** (DNS) resolves domain names to IP addresses (like a phone book)
- **Domain Registrars**: authority that signs unique domain names (GoDaddy, BlueGator)
- **State of Authority** (SOA): Contains for example name of server for zone, administrator of zone, default time-to-live (ttl = time a DNS record is cached), seconds of secondary name server should wait before checking for updates
- **Root Zone** controlled by Internet Assigned Numbers Authority (IANA)
- **Name Servers** (NS Records): used by tld servers to direct traffic to DNS server (which contains authoritative DNS records)
- **A records** (part of DNS record): “A” stands for IP Address
- **CNAME** (part of DNS record): resolves one domain name to another
- **Autonomous System** (AS) and Border gateway Protocol (BGP) info

Python libraries: `python-whois, dnspython, tldextract, ipaddress`

DNS Flow


Root NS
2
3
1
8
DNS
End User OS
DNS Resolver
4
5
TLD NS
9
6
7
Google Web server
Google NS

What makes them different?

URL Blocklist

amazon-sicherheitskunden-ueberpruefung.xyz
eclipsehotels.com/language/en-GB/eng.exe
bohicacapital.com/page
summerweb.net
ad.getfond.info
vdula.czystykod.pl/rxdjna2.html
svision-online.de/mgfi/administrator/
components/com_babackup/classes/fx29id1.txt

URL Allowlist

gurufocus.com/stock/...
dvdtalk.ru/review
333cn.com/zx/zhxw.html
made-in-china.com/special/led-lighting
google.com/u/0/112261544981697332354/posts
youtube.com/watch?v=Qp8MQ4shN6U
unesco.org/themes/education-sustainable-developm
thisisthefirst.com/page/5

Malicious URL Detection Features (Literature)

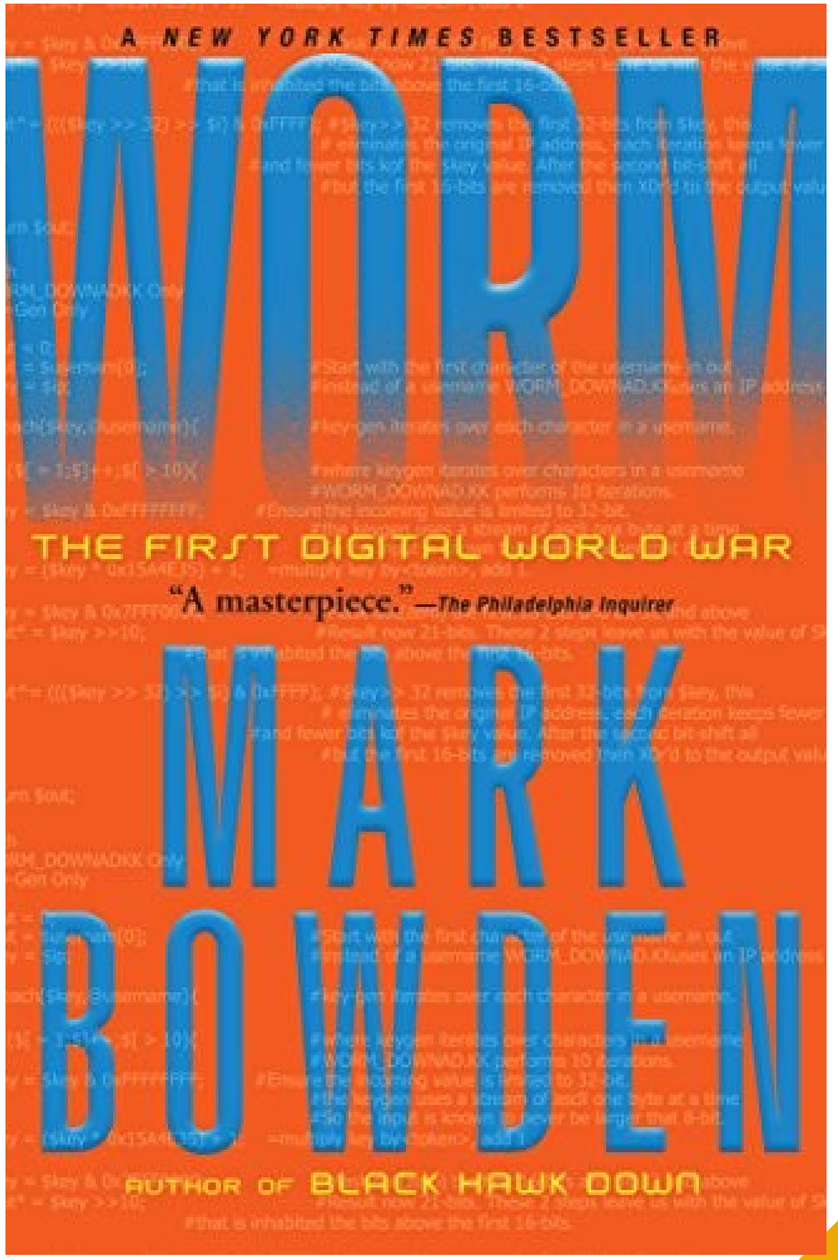
1. **BlackList Features**: BlackLists suffer from a high false negative rate, but can still be useful as machine learning feature.
2. **Lexical Features**: Capture the property that malicious URLs tend to "look different" from benign URLs. **Contextual information** such as the length of the URL, number of digits, lengths of different parts, entropy of domain name.
3. **Host-based Features**: Properties of web site host. **"Where"** the site is hosted, **"who" owns it** and **"how" it is managed**. API queries are needed (WHOIS, DNS records). Examples: Date of registration, the geolocations, autonomous system (AS) number, connection speed or time-to-live (TTL).
4. **Content-based Features**: Less commonly used feature as it requires **execution of web-page**. Can be not only be not safe, but also increases the computational cost. Examples: HTML or JavaScript based.

Sahoo et al. 2017: Malicious URL Detection using Machine Learning: A Survey

Domain Generating Algorithm (DGA) Detection

What is DGA?

- Domain Generating Algorithms (DGA) are algorithms which generate pseudo-random domains that are used for infected machines to communicate with the controller.
- The seeds on both the victim and command server are synchronized
- First seen with the Conficker worm.
- "Hello world" of cyber machine learning.


A NEW YORK TIMES BESTSELLER
WORM
THE FIRST DIGITAL WORLD WAR
"A masterpiece." —The Philadelphia Inquirer
MARK
BOWDEN
AUTHOR OF BLACK HAWK DOWN

Preparation In Class Exercise
ML Feature Engineering

Lexical Features

1. Length of URL
2. Length of domain
3. Count of digits
4. Entropy of domain
5. Position (or index) of the first digit
6. **Bag-of-words** for tld, domain and path parts of the URL

Data Set (Features and Target)

	url	isMalicious	domain	created	isIP	Length	LengthDor
56675	jeita.biz/w/google/drive/document.html?ssl=yes	1	jeita.biz	2012-04-11 17:08:19	0	46	9
73229	sosnovskoe.info/layouts/plugins/mailbox	1	sosnovskoe.info	2011-09-19 09:53:07	0	39	15
60112	teothemes.com/html/mp3pl/blue-preview.jpg	1	teothemes.com	2011-09-08 21:43:00	0	41	13
66946	kfj.cc:162/17852q	1	kfj.cc	2013-08-18 05:52:47	0	17	6
81906	verapdpf.info/db/6d1b281b5c4bbcfe3b99228680c232fa	1	verapdpf.info	2016-08-18 07:09:03	0	49	13

Features or X

Target or y

	isMalicious	isIP	Length	LengthDomain	DigitsCount	EntropyDomain	FirstDigitIndex	com	org	net	...	w	waset
73320	1	0	27	21	0	3.558519	0	0	1	0	...	0	0
30785	0	0	77	11	14	3.095795	22	1	0	0	...	0	0
60789	1	0	141	11	5	3.459432	103	0	1	0	...	0	0
19495	0	0	59	13	20	3.546594	31	1	0	0	...	0	0
45022	1	0	23	11	7	3.277613	13	0	0	0	...	0	0

Bag-of-Words

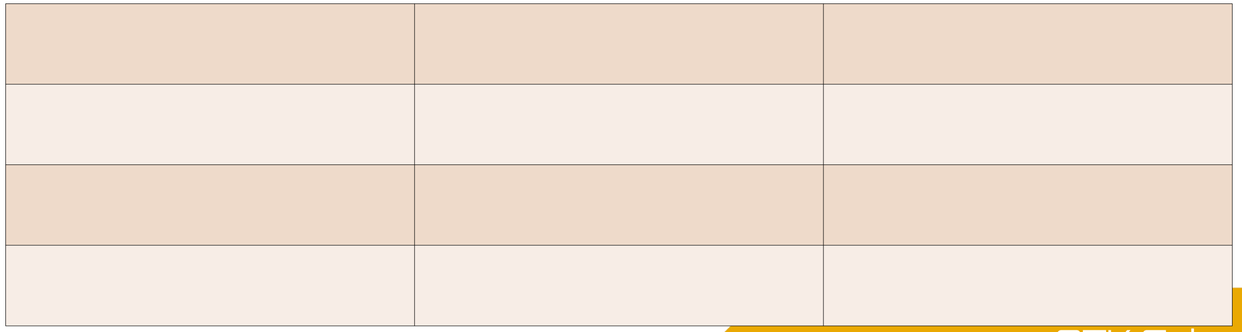
- Bag-of-words model: (Frequency of) occurrence of each word is used as a feature
- Sklearn's `CountVectorizer`: Convert a collection of text documents to a matrix of token counts

Simple Example: Imposing a vocabulary of `top_tlds=[‘.com’, ‘.de’, ‘.uk’]`

```
CountVectorizer_tlds = CountVectorizer(analyzer='word',
vocabulary=top_tlds)
CountVectorizer_tlds = CountVectorizer_tlds.fit(tlds)
matrix_tlds = CountVectorizer_tlds.transform(tlds)
```

Bag-of-words model fitting





http://scikit-learn.org/stable/modules/generated/sklearn.f

Preprocessing - an Art Work!

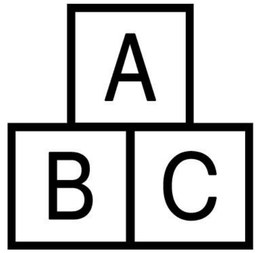
- Imputing missing values
- Scaling/Normalization
- One-Hot Encoding (Encoding categorical features)
- Embedding (e.g. word2vec)
- Binarizing (e.g. needed for Deep Learning multi-class target vector encoding)
- Encoding strings as int
- Dimensionality Reduction (e.g. PCA)
- Augmentation (e.g. tild/zoom images)
- Feature selection based on classifier
- Variance threshold

Data Types











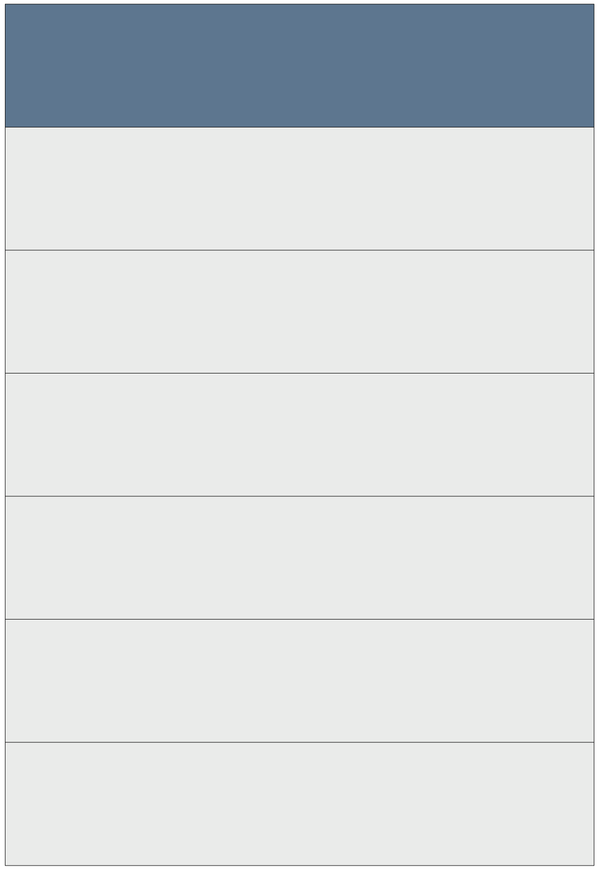
Primary Python libraries: `pandas`, `sklearn`, `scipy`

Missing Values

```
# using the most_frequent value
df['src_bytes'] = df['src_bytes'].fillna
# print the results
df['src_bytes'].value_counts().index[0])
# using the mean value
df['dst_bytes'] = df['dst_bytes'].fillna(df['dst_bytes'].mean())
```

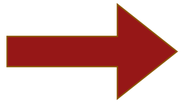
Categorical Variables
One-Hot Encoding

One Hot Encoding



One Hot Encoding

4 Categories



4 Columns with 1 when Category is True and delete original column!

One Hot Encoding

```
colors = ['Red', 'Red', 'Blue', 'Green', 'Yellow', ‘Red']
series_data = pd.Series( colors )
pd.get_dummies(series_data)
df = pd.get_dummies(df,
                    prefix=None,
                    prefix_sep=‘_’,


                    dummy_na=False,
                    columns=['protocol_type','flag'],
                    sparse=False
                    )
```

Better way: Feature Engine

- Feature Engine is a module which helps automate the complexities of feature engineering.
  - Missing Value Imputation
  - One Hot Encoding
  - Outlier Capping
  - More...
- Docs here: https://feature-engine.readthedocs.io/en/latest/
- Blog post: https://thedataist.com/when-categorical-data-goes-wrong/

Feature Scaling

Standard Scaling

Standardization - rescale so that mean = 0, std = 1

$$x_{scaled} = \frac{x - \mu}{\sigma}$$

```
scaled_feature = (feature - column_mean) /
standard_deviation
```

Standard Scaling

$$x_{scaled} = \frac{x - \mu}{\sigma}$$

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
scaler.fit(features)
features_scaled = scaler.transform(features)
```

Standard Scaling

```
original values:
[[  0.9   0.1  40. ]
[  0.3   0.2  50. ]
[  0.6   0.8  60. ]]
Mean of each column:
[  0.6  0.3667  50.]
SD of each column:
[ 0.2449  0.3091  8.165 ]
```

```
scaled values:
[[ 1.2247 -0.8627 -1.2247]
[-1.2247 -0.5392  0.    ]
[ 0.      1.4018  1.2247]]
Means of scaled data, per column:
[ 0. -0.  0.]
SD's of scaled data, per column:
[ 1.  1.  1.]
```

Notice that the mean of standard scaled data is zero and the StdDev is 1.

Min/Max Scaling

$$x_{scaled} = \frac{x - min}{max - min}$$

```
normed_feature = (feature - col_min) / (col_max -
col_min)
```

Min/Max Scaling

```
from sklearn.preprocessing import MinMaxScaler
minmax = MinMaxScaler()
minmax.fit(features)
features_scaled_minmax =
minmax.transform(features)
```

Min/Max Scaling

```
original values:
[[  0.9   0.1  40. ]
[  0.3   0.2  50. ]
[  0.6   0.8  60. ]]
Mean of each column:
[  0.6  0.3667  50.]
SD of each column:
[ 0.2449  0.3091  8.165 ]
```

```
scaled values:
[[ 1.      0.      0.    ]
[ 0.      0.1429  0.5   ]
[ 0.5     1.      1.    ]]

Means of scaled data, per
column:
[ 0.5    0.381  0.5  ]
SD's of scaled data, per column:
[ 0.4082  0.4416  0.4082]
```

Dealing with Imbalanced Classes

Dealing with Imbalanced Classes

- A lot of real-world security data will have very imbalanced targets.
- Fortunately, there is a library called imbalanced-learn which can assist.
- Imbalanced-learn provides a series of options to resample the data so that you have more balanced classes
- Docs available here: http://imbalanced-learn.org/

Dealing with Imbalanced Classes

```
from imblearn.over_sampling import RandomOverSampler
ros = RandomOverSampler(random_state=0)
X_resampled, y_resampled = ros.fit_resample(X, y)
```

Selecting Features

Should we use all of them?

How do we know which features to use and which to discard?

Before using automation, you should remove:

- Unnecessary features that are uninformative or repetitive
- Irrelevant features
- Duplicate observations

Selects *k* features according to the highest score

```
best_features =
SelectKBest(score_func=chi2,k=3).fit_transform(features,target)
```

Selects all features above a given threshold in the scoring function

```
best_features =
SelectPercentile(score_func=chi2,percentile=3).fit_transform(features,target
)
```

How do we know which features to use and which to discard?

Available Scoring Functions:

- **For regression:** f_regression, mutual_info_regression
- **For classification:** chi2, f_classif, mutual_info_classif

References:

http://scikit-learn.org/stable/modules/generated/sklearn.feature_selection.SelectKBest.html

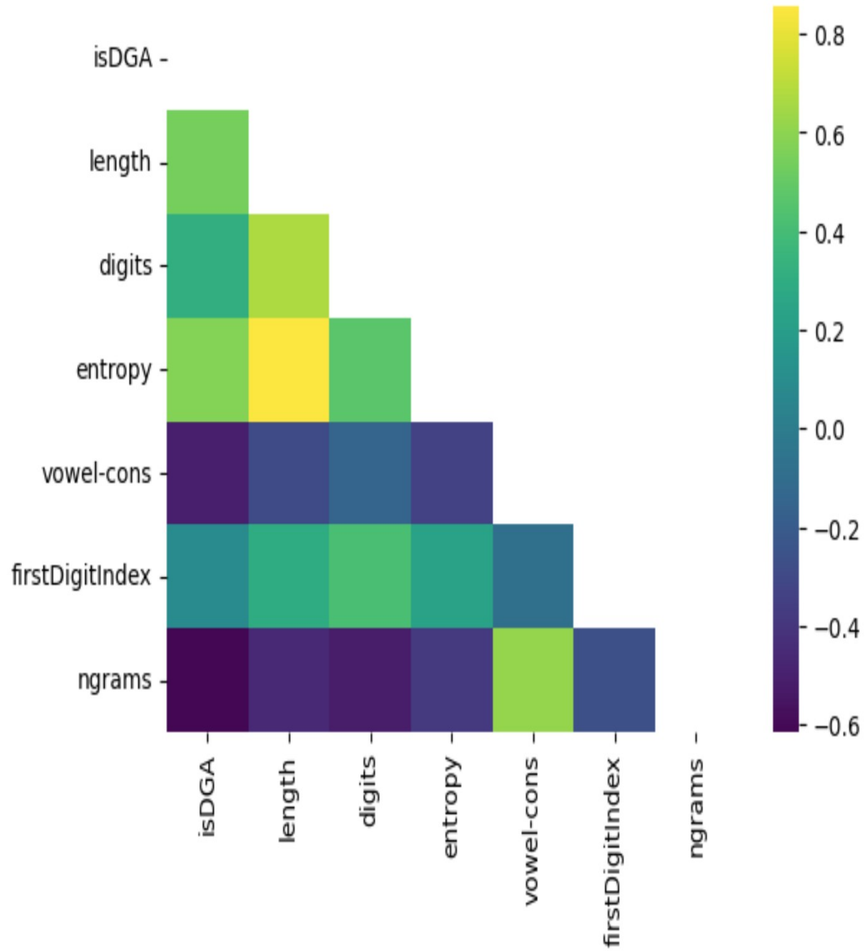
http://scikit-learn.org/stable/modules/feature_selection.html#univariate-feature-selection

How do we know which features to use and which to discard?

Visualize Them!!

Heatmap

```
sns.heatmap(corr_mat, mask=mask, cmap='viridis');
```



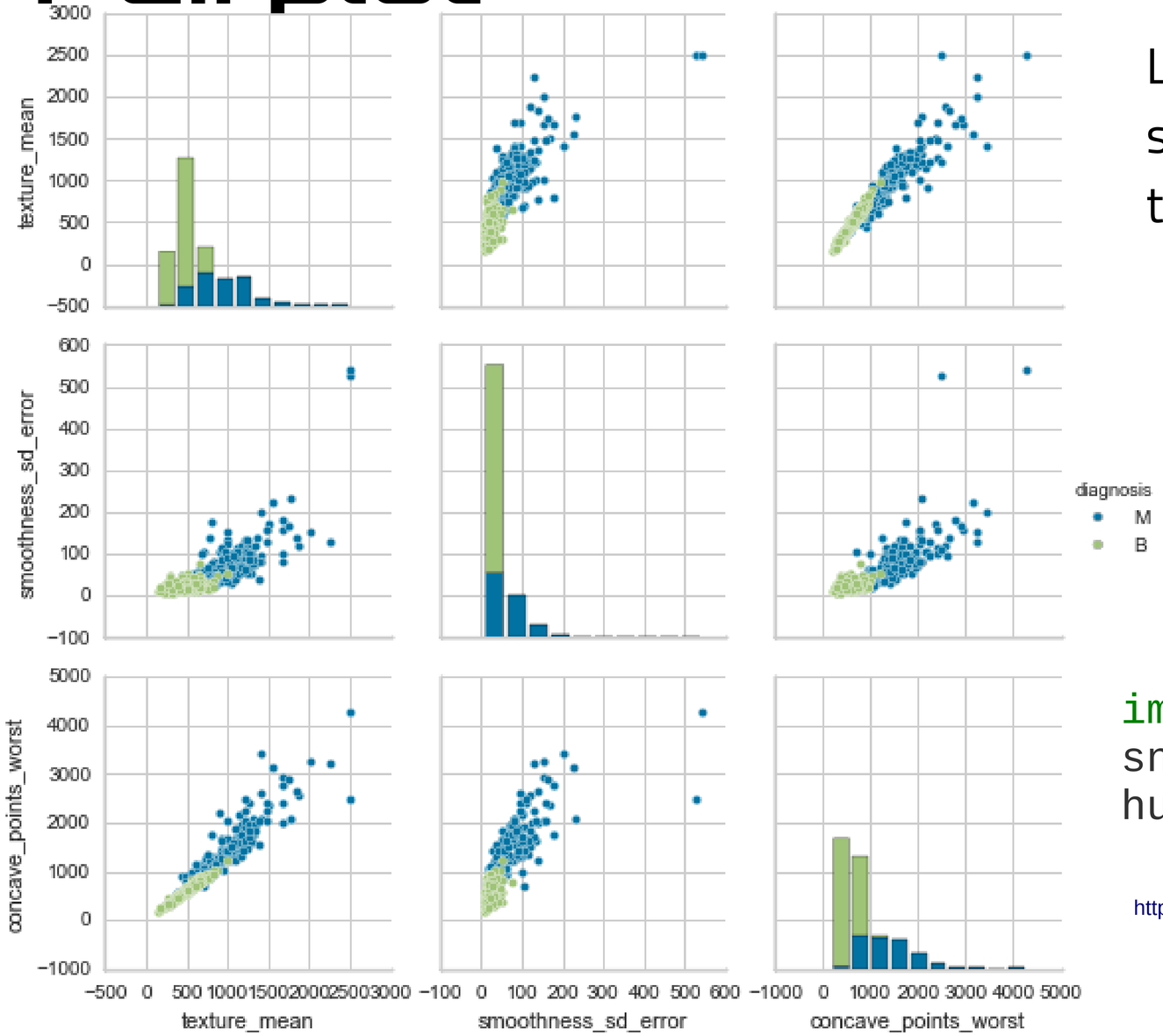
Now let's use a Seaborn pairplot as well. This will really show you which features have clear dividing lines between the classes. Docs are available h

Look at the correlations, covariance, etc between features by calculating the stats and plotting a heatmap of the matrix

```
import seaborn as sns
import pandas as pd
#Calculate correlations between features
corr_matrix = df_final.corr()
#Generate a mask for the upper
mask = np.triu(np.ones_like(corr_matrix,
dtype=bool))
#Plot the matrix as a heatmap
sns.heatmap(corr_mat, mask=mask, cmap='viridis')
```

strong correlation -> could mean similar information

Pairplot

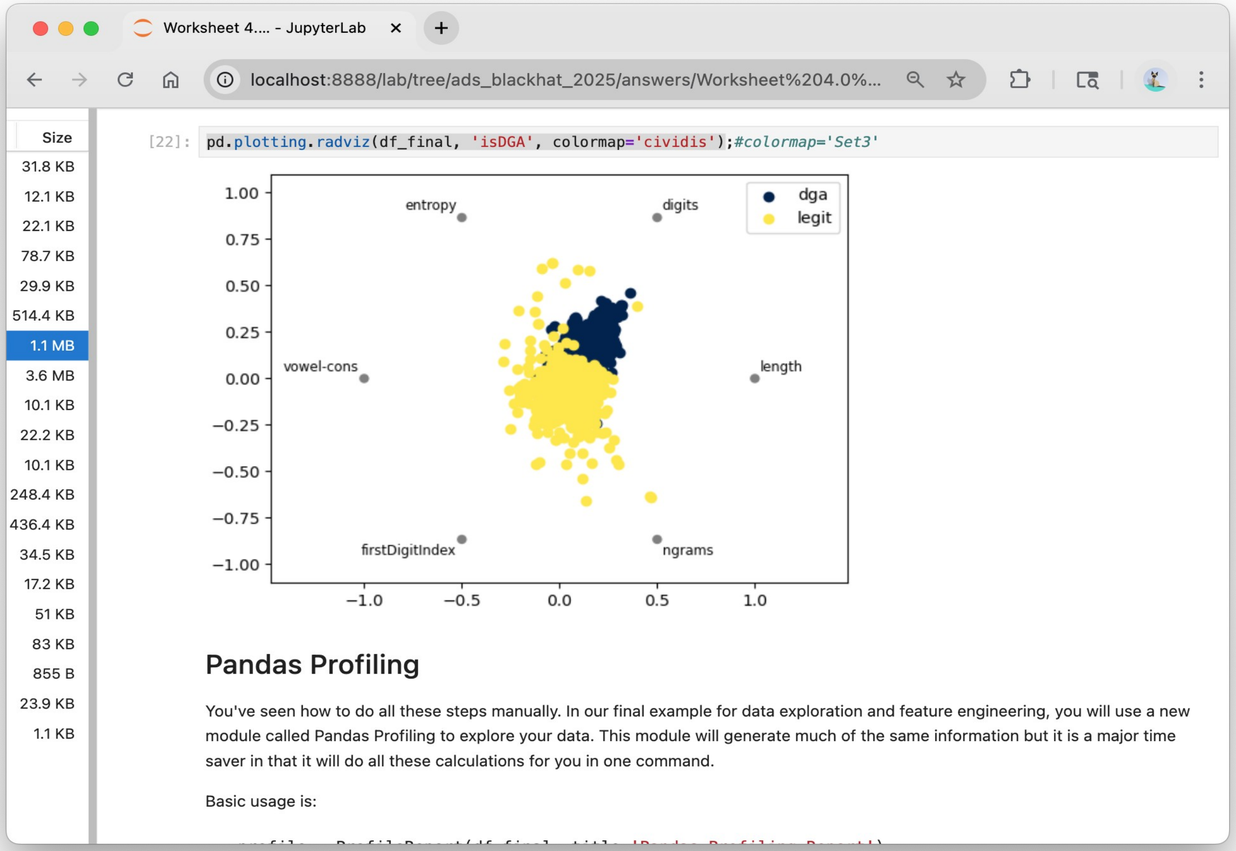


Look at the correlations between features with scatterplots of the targets, colored by the target class.

```
import seaborn as sns
sns.pairplot(<features>,
hue='<target>')
```

http://seaborn.pydata.org/generated/seaborn.pairplot.html

Radviz



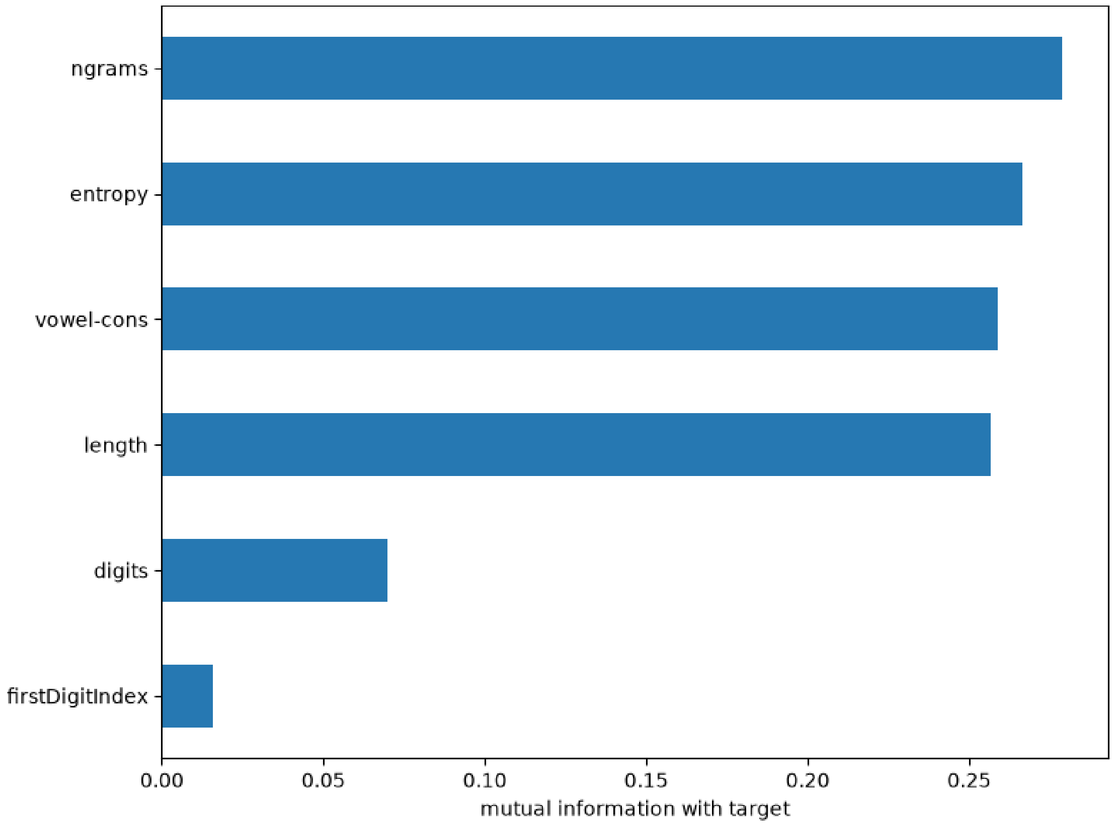
Veiw the contributions that each feature is making to the target via Pandas Radviz

```
import pandas as pd
pd.plotting.radviz(df,
‘<target_column’)
```

Mutual Information

- Captures non-linear relationships between the features and the target.
- Will help you identify features which are highly predictive that are not correlated at all.

Mutual Information Plot




```
from sklearn.feature_selection import
mutual_info_classif

features =
df_final_int.drop(columns="isDGA")

target = df_final_int["isDGA"].astype(int)


mi = mutual_info_classif(features, target)

mi = pd.Series(mi,
index=features.columns).sort_values()

mi.plot.barh(figsize=(8, 6))

plt.xlabel("mutual information with
target")

plt.tight_layout()
```

Do it all at once!

- FG-Data-Profiling (Formerly YData and Pandas-Profiling) is a module which can generate very thorough summaries of your datasets.
- Simple to use, but computationally intense...

```
from data_profiling import ProfileReport

profile = ProfileReport(df, title='DGA Feature Profiling
Report')
```

FG Data Profiling

- Pandas-
  summa
- Simple t

ıgh

Overview

Overview | Reproduction | Warnings 4

Dataset statistics

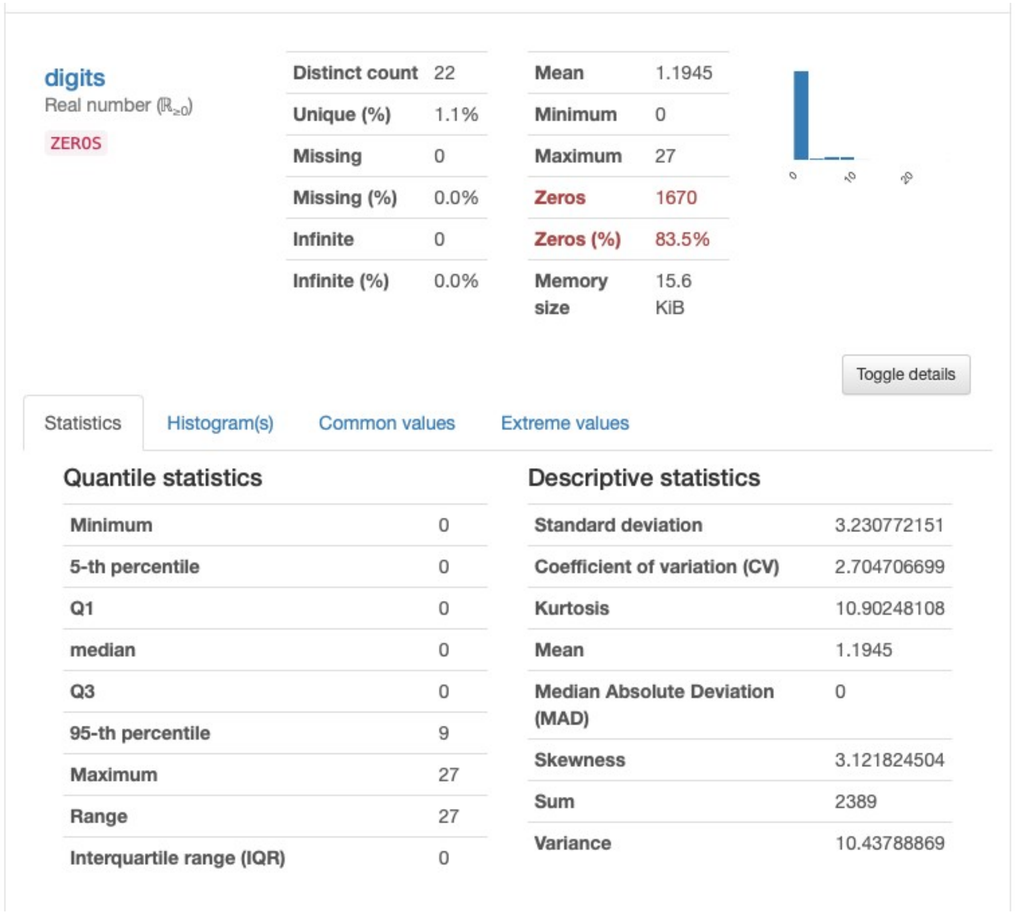
Number of variables	7
Number of observations	2000
Missing cells	0
Missing cells (%)	0.0%
Duplicate rows	2
Duplicate rows (%)	0.1%
Total size in memory	109.5 KiB
Average record size in memory	56.1 B

Variable types

NUM	6
BOOL	1

FG Data Profiling

- Pandas-Profiling i ery thorough summaries of you
- Simple to use, but



digits
Real number (ℝ≥0)
ZEROS

Distinct count	22
Unique (%)	1.1%
Missing	0
Missing (%)	0.0%
Infinite	0
Infinite (%)	0.0%

Mean	1.1945
Minimum	0
Maximum	27
Zeros	1670
Zeros (%)	83.5%
Memory size	15.6 KiB

Toggle details

Statistics | Histogram(s) | Common values | Extreme values

Quantile statistics

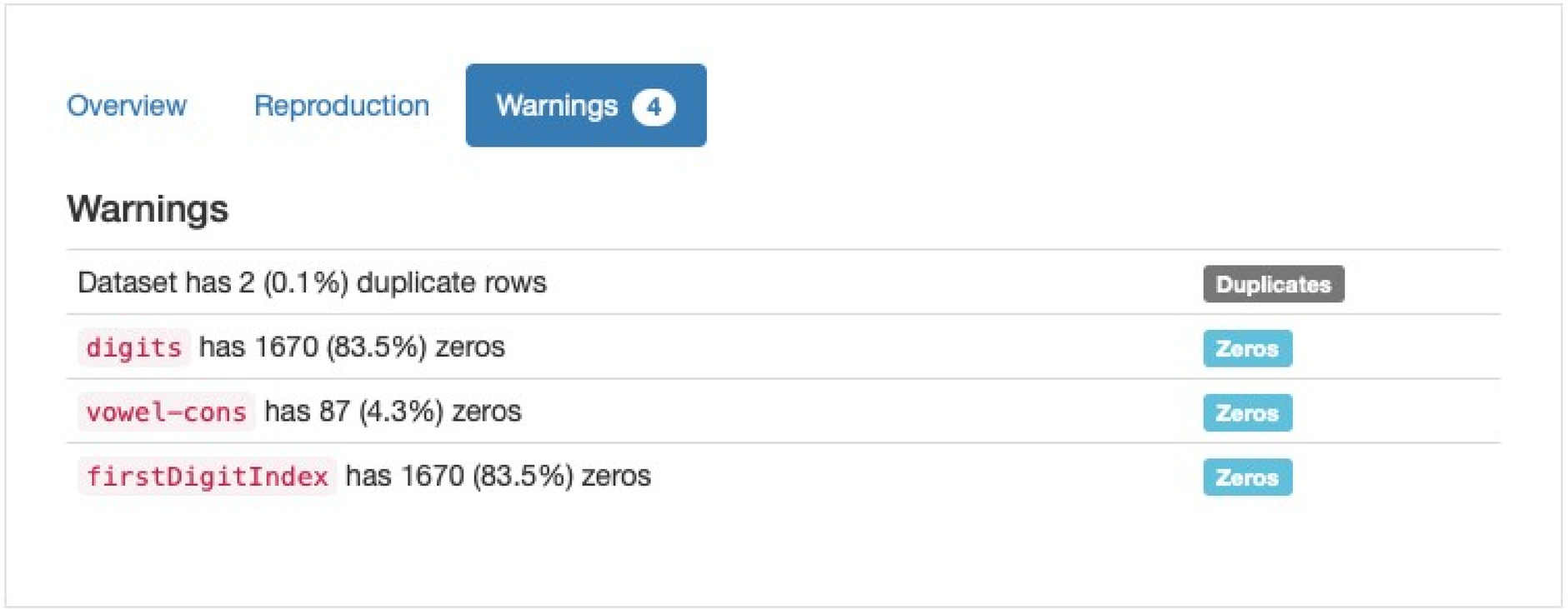
Minimum	0
5-th percentile	0
Q1	0
median	0
Q3	0
95-th percentile	9
Maximum	27
Range	27
Interquartile range (IQR)	0

Descriptive statistics

Standard deviation	3.230772151
Coefficient of variation (CV)	2.704706699
Kurtosis	10.90248108
Mean	1.1945
Median Absolute Deviation (MAD)	0
Skewness	3.121824504
Sum	2389
Variance	10.43788869

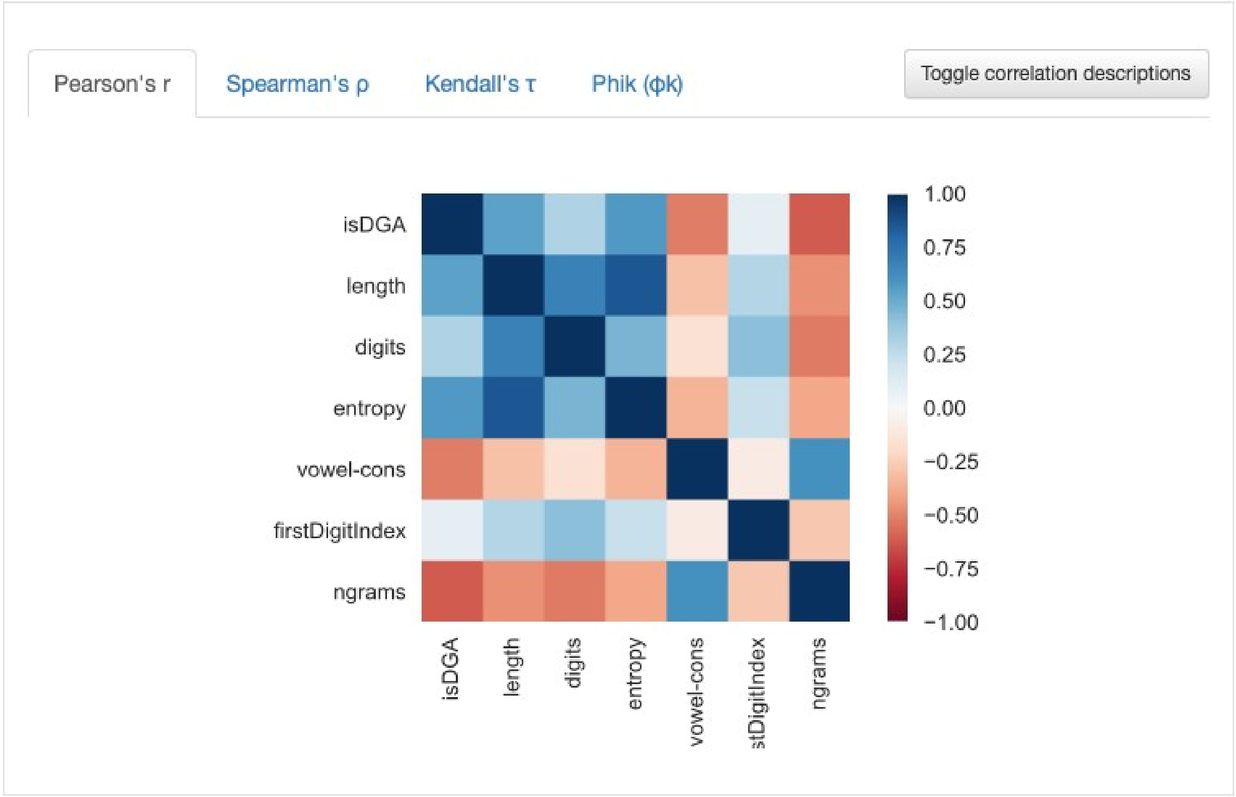
FG Data Profiling

- Pan
  sum
- Sim



FG Data Profiling

- Pandas-Profi ıorough summaries of
- Simple to use



In Class Exercise

Please take 45 minutes and complete

Worksheet 3.1 - Feature Engineering

Questions?